

# Project 8: Create EBS Snapshots and Restore an EC2 Instance

## Objective

To create an Amazon EBS snapshot of an EC2 instance volume and restore the data by creating a new volume from the snapshot and attaching it to an EC2 instance.

## Project Description

This project demonstrates how Amazon Elastic Block Store (EBS) snapshots can be used to back up EC2 instance data and restore it when required. The project covers identifying an EBS root volume, creating a snapshot, restoring the snapshot to a new volume, attaching it to an EC2 instance, verifying the restored data, and finally cleaning up all AWS resources to avoid unnecessary charges.

## AWS Services Used

1. Amazon EC2
2. Amazon EBS
3. Amazon EBS Snapshots
4. AWS Budgets

## Architecture Overview

An EC2 instance running Amazon Linux 2023 was launched using a free-tier eligible instance type. The root EBS volume attached to the instance was backed up using an EBS snapshot. A new EBS volume was created from this snapshot and attached to the same EC2 instance as a secondary volume to verify data restoration.

## Step-by-Step Implementation

### Step 1: Launch EC2 Instance

- Launched a free-tier eligible EC2 instance (t3.micro)
- Amazon Linux 2023 AMI selected

- Verified that the root device type is EBS

## **Step 2: Identify the Root EBS Volume**

- Navigated to EC2 → Volumes
- Identified the root EBS volume attached to the instance

## **Step 3: Create Test Data**

- Connected to the EC2 instance
- Created a test directory and file on the root volume
- Added sample text to verify snapshot restoration

## **Step 4: Create EBS Snapshot**

- Created a snapshot of the root EBS volume
- Verified snapshot completion status

## **Step 5: Create Volume from Snapshot**

- Created a new EBS volume from the snapshot
- Ensured the volume was created in the same Availability Zone as the EC2 instance

## **Step 6: Attach Restored Volume**

- Attached the restored volume to the EC2 instance as a secondary device
- Verified the attachment was successful

## **Step 7: Mount and Verify Restored Data**

- Mounted the restored volume on the EC2 instance
- Used the `nouuid` mount option due to XFS filesystem UUID duplication
- Verified that the test file and data were successfully restored

## **Step 8: Cleanup Resources**

- Unmounted the restored volume
- Detached and deleted the restored EBS volume
- Deleted the snapshot

- Terminated the EC2 instance
- Verified no active resources remained to avoid charges

## Challenges Faced and Resolution

### XFS UUID Conflict:

When mounting the restored volume, a filesystem UUID conflict occurred. This was resolved using the `nouuid` mount option, which allows mounting cloned XFS filesystems safely.

## Outcome

- Successfully restored data from an EBS snapshot
- Verified data integrity after restoration
- Demonstrated proper AWS cost management through complete cleanup

## Conclusion

This project demonstrated the importance of EBS snapshots for data backup and recovery in AWS. It also highlighted real-world scenarios such as filesystem conflicts and emphasized best practices for safe resource cleanup and cost optimization.

## Screenshots

All screenshots for each step are included in the submission as evidence of successful implementation.